

Algorítmica y programación I

LABORATORIO N°5 Sistema de
restaurante



ORDINA O-RA
Stimato 

Alumnos: Tomas Jaurena, Imanol Limarieri,
German Rosales, Nahuel Barria

RESUMEN:

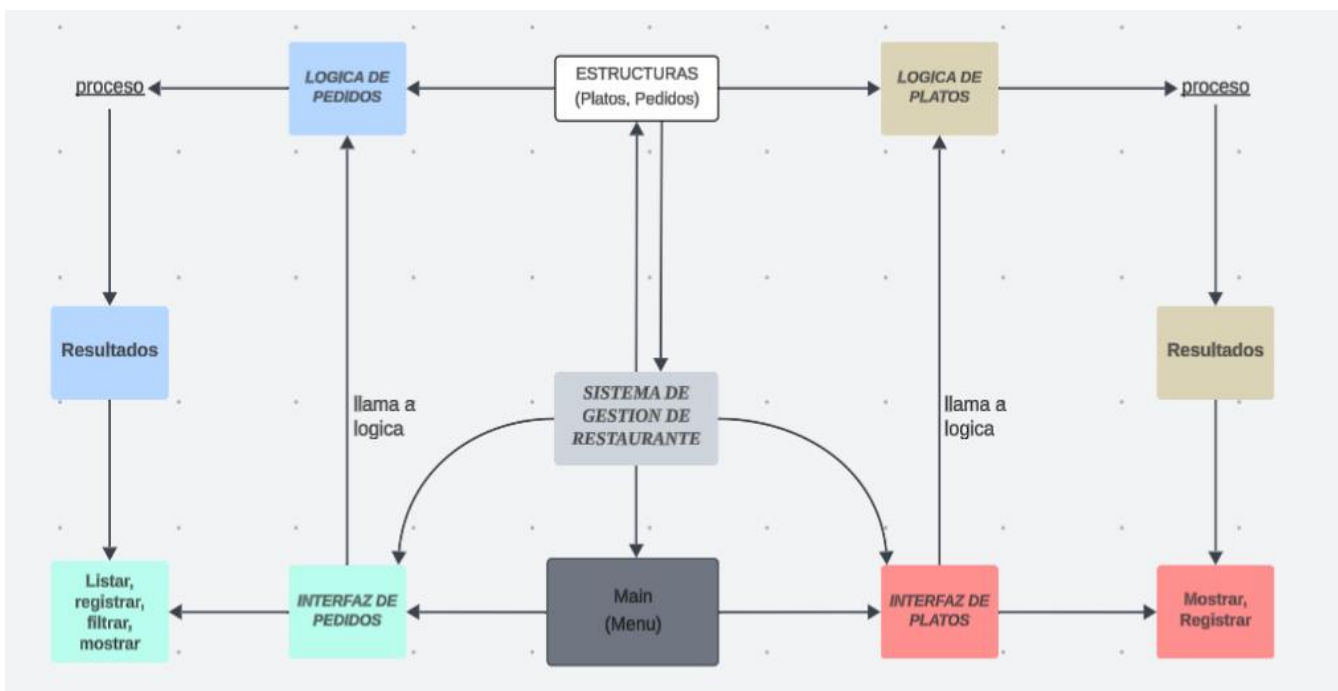
Este laboratorio consiste en el desarrollo de un sistema de gestión de pedidos, menús y cliente en lenguaje C. Se trabajará en tres incrementos parciales que permitirán aplicar conceptos de estructuras, algoritmos, búsquedas, manejo de archivos, punteros y diseño de una interfaz básica.

Primero nos encargamos de planificar cómo se iba a trabajar. Se planteó un desarrollo en base a una arquitectura en capas, basado en dividir al código en dos capas, donde una se encargaría de la interfaz del programa y la otra de la parte lógica del mismo. Además dividimos el código en dos entidades, cada una relacionada con una estructura propia, las cuales serían los platos y los pedidos del restaurante.

Implementamos el uso de archivos (de tipo texto) para almacenar y gestionar los platos del restaurante, pedidos realizados y un resumen diario del establecimiento.

Dentro del programa en base a un menú, el operador del sistema puede elegir varias opciones de las cuales podrá:

- Gestionar pedidos (agregar, listar, completar, filtrar, buscar)
- Gestionar platos (actualizar, verificar, buscar, agregar)
- Realizar un reporte general acorde a lo realizado durante la jornada



ESTRUCTURAS:

```
struct Plato
{
    int id;           // Identificador único del plato
    char nombre[50];  // Nombre del plato
    float precio;     // Precio del plato
    int stock;        // Cantidad disponible
};
```

```
struct Pedido
{
    int id;           // Identificador único del pedido
    char cliente[100]; // Nombre del cliente
    int idPlato;      // ID del plato pedido
    int cantidad;     // Cantidad solicitada
    float total;      // Precio total del pedido
    int activo;       // 1 = activo, 0 = completado
};
```

VARIABLES MÁS IMPORTANTES:

```
struct Pedido pedidos[100]    // arreglo que permite guardar los pedidos recibidos
struct Plato platos[100]      // arreglo que permite guardar los platos disponibles
```

```
int cantPlatos    //indica la cantidad de platos y además como índice de platos
```

```
int cantPedidos   //indica la cantidad de pedidos y además como índice de pedidos
```

```
int opcion        //indicador para seleccionar el caso en el menú
```

INCREMENTO 1:

En este incremento nos basamos más que nada en plantear las bases del programa y de las estructuras que íbamos a utilizar en el mismo.

Funciones Principales:

- Registrar Pedido:

```
void registrarPedidoInterfaz(struct Pedido pedidos[], int
*cantPedidos, struct Plato platos[], int *cantPlatos);
```

Esta función se encarga de gestionar desde la interfaz el proceso de registrar un nuevo pedido. Su objetivo es pedir los datos al usuario, validar el stock, y luego llamar a la lógica correspondiente. Para realizarse, recibe como parámetros el arreglo de pedidos, la cantidad de pedidos recibidos, el arreglo de platos y la cantidad de platos disponibles.

Pasos que realiza:

- Solicita el nombre, lo guarda en `cliente` y lo pasa a minúsculas con `convertirminus`.
- Dentro de un bucle `do-while`, muestra los platos disponibles con la función `(mostrarCartaInterfaz)` y pide:
 - El ID del plato
 - La cantidad solicitada del mismo
- Llama a `verificarStock` para confirmar si hay suficiente stock.
 - Si falta stock, informa el error y repite el bucle.
 - Si hay stock, sale del bucle.
- Llama a `registrarPedidoLOGIC`, pasando los datos validados:
 - El arreglo de pedidos
 - El puntero a la cantidad de pedidos
 - El arreglo de platos
 - El id del plato seleccionado
 - La cantidad elegida

- Accede al último pedido ingresado y muestra su total a pagar.

```
void registrarPedidoLOGIC (struct Pedido pedidos[], int *cantPedidos,  
struct Plato platos[], int idPlato, int cantidad, char cliente[100]);
```

Es la función que se encarga de tomar los datos que recibe de `registrarPedidoInterfaz` y almacenarlos en un registro de tipo Pedido dentro del arreglo pedidos. Luego de finalizar la carga se llama a `actualizarStock` para modificar las unidades disponibles del plato seleccionado por el pedido, además de incrementar a la variable cantPedidos.

- Registrar Plato:

```
void registrarPlatoInterfaz(struct Plato platos[], int *cantPlatos);
```

Esta función se encarga de agregar un nuevo plato al menú, validando los datos ingresados por el usuario desde la interfaz.

Pasos que realiza:

1. Solicita:

- Nombre del nuevo plato
- Precio
- Stock disponible

2. Llama a `verificarPlato`.

Si la función devuelve 1, significa que el plato ya está en el menú. Muestra el mensaje y sale de la función.

3. Valida el precio ingresado con `comprobacionNum` para verificar que sea:

- numérico.
- mayor que 0.

Si el valor es inválido:

- Muestra un mensaje de error.
- Repite la solicitud del precio

El ciclo solo termina cuando el precio ingresado es válido.

4. Valida el stock ingresado de forma similar al precio.

5. Llama a `registrarPlatoLogica`

Donde guarda el plato en el arreglo usando los datos ya validados.

6. Incrementa:

```
*cantPlatos = (*cantPlatos + 1);
```

```
void registrarPlatoLogica(struct Plato *platos, char nombre[100], float
precio, int stock, int id);
```

Recibe los datos ya validados (nombre, precio, stock, id) y los guarda dentro del struct `Plato` al que apunta el puntero `platos`.

- Mostrar Carta:

```
void mostrarCartaInterfaz(struct Plato platos[], int cantPlatos)
```

La función llama a la parte lógica, `MostrarCartaLogica`; la cual devuelve un puntero de tipo struct almacenado en “`struct Plato *disponibles`”. Dicho puntero contiene los platos disponibles, es decir, que tengan un stock mayor a 0. Luego los imprime de forma ordenada y prolija mediante un ciclo iterativo (for).

Al final libera la memoria dinámica que la parte lógica había reservado:

```
free(disponibles);
```

```
struct Plato *MostrarCartaLogica(struct Plato platos[], int
cantPlatos, int *cont)
```

Esta función es la encargada de filtrar los platos con stock mayor a 0. Para ello se crea un arreglo de registros asignándole memoria dinámica, referenciado con un puntero de tipo estructura.

```
struct Plato *disponibles = malloc(sizeof(struct Plato) *
(size_t)disponibleCount);
```

Esto crea un arreglo dinámico donde se almacenen los platos que tienen stock luego dentro de la función se verifica si se reservó memoria dinámica de forma correcta o no con un NULL.

Luego recorre todos los platos registrados y un contador aumentará solo cuando el stock del mismo es mayor a 0. Dicho contador se usará como límite de una estructura iterativa de vueltas fijas que utilizará la capa de interfaz.

```
if (cont)
{
    *cont = disponibleCount;
}
```

Se implementa un ciclo iterativo que recorre todos los platos del arreglo original y, cada vez que encuentra uno cuyo stock es mayor a cero, lo copia al nuevo arreglo dinámico. Para colocar cada plato en la posición correcta dentro del nuevo arreglo, utiliza la variable **j**, que empieza en 0 y se incrementa solo cuando se copia un plato válido. De esta manera, el arreglo dinámico queda formado únicamente por los platos que tienen stock disponible, sin huecos y en el mismo orden en que aparecían en la lista original.

```
int j = 0;

for (int i = 0; i < cantPlatos; i++)
{
    if (platos[i].stock > 0)
    {
        disponibles[j++] = platos[i]; // copia del struct
    }
}
```

```
}  
  
}
```

INCREMENTO 2:

En este incremento nos basamos en implementar algoritmos de búsqueda y ordenamiento, además realizamos filtros tanto en platos como pedidos utilizando registros auxiliares asignados con memoria dinámica. Por último implementamos el uso de archivos para importar y exportar datos.

Funciones Principales:

- Listar Pedidos Activos

```
void ListarPedidosActivosInterfaz(struct Pedido pedidos[], int  
cantPedidos)
```

Esta función se utiliza para mostrar los pedidos que están en proceso de elaboración, para ello esta recibe un puntero tipo struct y la cantidad de pedidos realizados hasta el momento.

Luego llama a `listarPedidosActivosLogica` que devuelve un puntero a un arreglo de registros creado dinámicamente y lo guarda en un puntero tipo struct, dicho puntero contiene el filtrado de los pedidos para que se impriman. Luego libera la memoria que fue asignada de forma dinámica

```
struct Pedido *listarPedidosActivosLogica(struct Pedido pedidos[], int  
cantPedidos, int *cantActivos)
```

Esta función se encarga de filtrar los pedidos que hay activos utilizando memoria dinámica, devolviendo un puntero tipo struct.

```
struct Pedido *activos = malloc(sizeof(struct Pedido) *  
(size_t)contador );
```


Luego dentro de un bucle For busca aquellos pedidos que cumplan con la condición de estar activos, y los coloca en el arreglo `activos`.

- Funciones de ordenamiento:

```
void ordenarPorPrecio(struct Pedido pedidos[], int cantPedidos)
```

Esta función utiliza un método de ordenamiento (burbuja) para ordenar el arreglo de pedidos acorde a el precio del mismo.

```
void OrdenarPorId(struct Pedido pedidos[], int inicio, int fin)
```

Esta función utiliza un método de ordenamiento (merge sort) para ordenar el arreglo de pedidos acorde al ID de los mismos.

- Funciones de búsqueda:

```
void busquedaIdInterfaz(struct Pedido pedidos[], int cantPedidos)
```

```
int busquedaIdLogica(struct Pedido pedidos[], int cantPedidos, int valor)
```

- La parte de interfaz solicita al usuario que ingrese un id del pedido que desea buscar y se lo pasa a la función lógica como parámetro junto a el arreglo de los pedidos y la cantidad de pedidos realizados hasta el momento. la parte lógica implementa un algoritmo de búsqueda binaria en base al número de id del pedido, una vez que lo encuentre en el arreglo devuelve la posición del índice en donde se encuentra a la interfaz y este imprime todos los datos relacionados al mismo, en caso de no ser encontrado retorna un valor (-1) e indica por pantalla que el pedido no existe.

```
void filtrarPedidoMontoInterfaz(struct Pedido pedidos[], int cantPlatos)
```

```
int filtrarPedidoMontoLogica(struct Pedido pedidos, int monto)
```

- La parte de interfaz solicita al usuario que ingrese un monto para buscar, luego se pasa a la parte lógica el monto ingresado y pasa cada registro por separado verificando si el monto ingresado es igual al total de un pedido, en caso de ser iguales se devuelve un valor (1) como verdadero (se encontró) y en la parte de interfaz se imprimen todos los datos relacionados al mismo, en caso de no ser encontrado la parte lógica devuelve un valor (0) como falso indicando que no se encontró en el arreglo e indicando por un mensaje en pantalla.

```
void filtraPedidoClienteInterfaz(struct Pedido pedidos[], int cantPedidos)
```

```
int filtraPedidoClienteLogica(struct Pedido pedido, char cliente[])
```

- La parte de interfaz solicita al usuario que ingrese el nombre de un cliente para buscar, luego se pasa a la parte lógica el nombre ingresado verificando si el nombre ingresado es igual al nombre de un cliente del arreglo, en caso de ser iguales se devuelve un valor (1) como verdadero (se encontró) y en la parte de interfaz se imprimen todos los datos relacionados al cliente, en caso de no ser encontrado la parte lógica devuelve un valor (0) como falso indicando que no se encontró en el arreglo e indicando un mensaje en pantalla.

- Funciones de archivos

```
void importarPlatos(struct Plato platos[], int *cantPlatos)
```

- Esta función extrae la información de los platos que se encuentra en el archivo **"Platos.txt"**, los cuales están escritos en el siguiente formato:

'1,milanesa con papas fritas,9500.000000,25'

Luego en un bucle while tomo cada línea del archivo hasta que llegue al fin del mismo. dentro de cada iteración se coloca la línea tomada del archivo en un arreglo auxiliar y se utiliza la función `strtok`, la cual permite seleccionar un arreglo y un delimitador, en este caso sería `,`, creando sub arreglos con la información fragmentada de los platos que coincide con los campos del registro de plato, luego la información es colocada en los registros.

```
char *tok = strtok(cadena, ","); //tok permite delimitar con la coma puesto que es lo que permite separarlos campos en los platos
```

```
void exportarPlatos(struct Plato platos[], int cantPlatos)
```

- La función recibe como parámetros el arreglo de platos y la cantidad de platos y abre el archivo anteriormente mencionado **"Platos.txt"** en modo de escritura **"w"**, luego mediante un ciclo iterativo se van mostrando por pantalla todos los platos que estaban anteriormente guardados en el archivo **"Platos.txt"** dando formato en torno a la estructura **"platos"**.

```
void exportarPedidos(struct Pedido pedidos[], int cantPedidos)
```

- La función recibe como parámetros el arreglo de pedidos y la cantidad de pedidos y abre el archivo **"Pedidos.txt"** en modo de escritura **"w"**, luego mediante un ciclo

iterativo se van mostrando por pantalla todos los pedidos que estaban anteriormente guardados en el archivo "**Pedidos.txt**" dando formato en torno a la estructura "pedidos".

INCREMENTO 3:

Funciones Principales:

- Función de registro final:

```
void registro(struct Plato platos[], struct Pedido pedidos[], int cantPedidos)
```

→ Es la encargada de mostrar un registro detallado de la actividad del restaurante hasta ese momento. dentro de este registro se encuentra detallado:

1. El total de pedidos realizados, para lo cual usa la variable cantPedidos.
2. La cantidad de pedidos que se encuentran activos, para ello utiliza la función:

```
int contarPedidos(struct Pedido pedidos[], int cantPedidos, int modo)
```

3. La cantidad de pedidos que se completaron, para la cual utiliza:

```
int contarPedidos(struct Pedido pedidos[], int cantPedidos, int modo)
```

4. El total de dinero recaudado en el día, que utiliza la función:

```
float platita(struct Pedido pedidos[], int cantpedidos)
```

5. El plato que aparece en la mayor cantidad de pedidos distintos, donde se utiliza:

```
int elMasPedido(struct Pedido pedidos[], int cantPedidos, int *índice)
```

→ El objetivo principal de esta función es la de contabilizar las veces que se piden los distintos platos en todos los pedidos realizados. Para ello utiliza un arreglo `arrplatos` previamente inicializado con todos sus valores en cero. Luego recorre todos los pedidos enfatizando en el campo `idPlato`, donde ese valor funciona como índice para el arreglo, donde incrementa en uno el valor de esa posición. Después de recorrer todos los pedidos, se recorre el arreglo buscando el valor más alto, y

guardando su índice que se le asigna al puntero suministrado. Por último la función devuelve dicho valor máximo.

6. El cliente que más pedidos ha realizado, y el monto total que gastó, este procedimiento es completamente delegado a las funciones:

```
void clienteMayorGastoInterfaz(struct Pedido pedidos[],  
int cantPedidos)
```

- Esta función es la encargada de imprimir lo relacionado con el cliente con mayor gasto para eso utiliza la parte lógica y esta última devuelve el nombre del cliente y el gasto total del mismo

```
void clienteMayorGastoLogica(struct Pedido pedidos[], int  
cantPedidos, char clienteMax[100], float *gastoMax);
```

- Inicializa una matriz de caracteres en la que guarda los nombres de los clientes, donde cada fila son los id y cada columna son los nombres de los clientes (esto está propuesto de esta manera para evitar que los nombres se repitan). Se inicializa otro arreglo en donde en cada índice se guardan los gastos de cada cliente (inicialmente hardcodeado en 0) luego mediante un ciclo iterativo recorreremos los registros de pedidos acorde a la cantidad de pedidos.
- Mediante una variable usada como boole (es decir 0 para falso, 1 para verdadero) se ingresa a otro ciclo iterativo anidado dentro del anteriormente mencionado el cual es usado para corroborar si el cliente está dentro del arreglo, a través de una comprobación se verifica si el cliente está o no dentro del arreglo, en caso de estar:
 - A. En caso de estar se suman los pedidos realizados por el mismo cliente utilizando el arreglo de gastos y se cambia la variable boole (encontrado) de 0 a 1 confirmando que el cliente está en el arreglo, luego se sale del for.
 - B. En caso de NO estar se agrega el cliente a la matriz, se suma la cantidad de clientes en +1 ya que se agrega uno nuevo y se le asigna en el arreglo de gastos el monto de este nuevo cliente.
- Una vez termina de recorrer la matriz y se asegura de que todos los clientes estén asignados en su respectivo índice se termina por verificar cual es el cliente el cual tuvo mayor gasto, en un ciclo iterativo y mediante una comprobación se verifica cada índice del arreglo de gastos, inicialmente la primera comprobación se da con una variable temporal inicializada en 0, en caso de que el gasto sea mayor (que en

la primera comprobación siempre lo será) se guardará en la variable temporal el índice del arreglo de gastos que contiene el mayor gasto para luego repetirse el proceso buscando el mayor gasto en todo el arreglo. Ya cuando finaliza este proceso se guarda el mayor gasto en un puntero y el nombre del cliente el cual tuvo el mayor gasto, estos dos valores se devuelven a la interfaz para mostrarse por pantalla.

INSTRUCTIVO:

Al iniciar el programa se presenta el siguiente menú de opciones:

```
++-----++
||          SISTEMA DE RESTAURANTE          ||
++-----++
||
||  Estimado, seleccione una opcion:
||
||  [1] Mostrar menu
||  [2] Mostrar reporte
||  [3] Mostrar pedidos activos
||  [4] Buscar pedido por cliente
||  [5] Buscar pedido por monto
||  [6] Buscar pedido por ID
||  [7] Registrar pedido
||  [8] Registrar plato
||  [9] Marcar pedido completado
||  [10] Exportar pedidos
||  [0] Salir del menu
||
++-----++
```

1. Mostrar menú: Muestra en pantalla todos los platos disponibles.
2. Mostrar reporte: Muestra un reporte del día.
3. Mostrar pedidos activos: Muestra los pedidos que todavía no fueron entregados.
4. Buscar pedido por cliente: Busca un pedido por el nombre del cliente, para ello debe indicarse el nombre del cliente que se desea buscar.
5. Buscar pedido por monto: Busca un pedido por un monto exacto , para ello se debe de indicar el monto a buscar.
6. Filtrar pedido por id: Busca un pedido por su id, para ello se debe de indicar el id a buscar.
7. Registrar pedido: Permite agregar un pedido de un cliente, se deberá ingresar nombre, Id del plato y cantidad.
8. Registrar plato: Permite agregar un nuevo plato al menú, al ingresar en esta opción se debe de colocar: nombre del plato,cantidad disponible, precio. Al momento de ingresar el nombre el sistema validará que ya no se encuentre ingresado, mientras que al ingresar la cantidad y precio se validará que no sean valores negativos o cero.

9. Marcar pedido completado: Permite pasar el estado del pedido de activo a completado, para ello debe de indicar el id del pedido a completar.
10. Exportar pedidos: Crea un archivo de texto con el listado de todos los pedidos cargados al sistema hasta ese momento
0. Salir del programa.